

DIGITAL IMAGE PROCESSING
(CS 406)

Class Activity 4: Working with Histograms

Histogram Computation, Equalization & CLAHE using Python (OpenCV & Matplotlib)

Instructor: Dr. Jameel Ahmad
Department: Computer Science & Engineering
Total Marks: 100

Term: Summer 2026
Submission Date: September 23, 2026
Source Notebook: `cas/ca_04/ca_04.ipynb`

Student Name	Reg. No.	Assigned Roles & Tasks
Muhammad Umer	F2023266912	Role A – Data & Histogram Analyst (Task 1); Role C – Adaptive Enhancement Engineer (Task 3); co-author of Task 5
Haseeb Ullah	F20232661009	Role B – Global Enhancement Engineer (Task 2); Role D – Color Space Specialist (Task 4); co-author of Task 5

Note: our group has 2 active members. Each member absorbed two of the four rotating roles specified in the CA-4 brief so that all five tasks were completed in full and both members can independently explain every task, as required for the in-class viva.

Executive Summary

This report documents a complete, hands-on tour of histogram-based image enhancement in OpenCV: computing and interpreting 1-D and 2-D histograms (`plt.hist`, `cv2.calcHist`); applying and diagnosing the artifacts of *global* histogram equalization (`cv2.equalizeHist`); implementing and tuning *Contrast-Limited Adaptive Histogram Equalization* (CLAHE) across its two key hyperparameters; correctly enhancing color imagery by operating on an intensity-only channel (HSV’s V or LAB’s L) instead of R/G/B independently; and building a small ROI-based selective-enhancement tool driven by `cv2.selectROI()` and binary masking. All results in this report were generated by executing `cas/ca_04/ca_04.ipynb` end-to-end; every figure below is a real output of that run, not a mock-up.

Contents

1	Setup, Environment & Dataset Construction	3
1.1	Dataset	3
2	Task 1 – Histogram Fundamentals (Student A, 20 marks)	5
2.1	Loading and displaying the grayscale image	5
2.2	Bin-count resolution: <code>bins=256</code> vs. <code>bins=10</code>	5

2.3	Histogram of the binary image	5
2.4	Overlaid B/G/R channel histograms	6
2.5	2-D joint histogram of B and G (cv2.calcHist)	6
3	Task 2 – Global Histogram Equalization (Student B, 20 marks)	7
3.1	Applying cv2.equalizeHist()	7
3.2	Histograms before and after	7
3.3	Annotated artifact regions	8
3.4	Discussion	9
4	Task 3 – CLAHE Implementation & Parameter Tuning (Student C, 20 marks)	10
4.1	Basic CLAHE object	10
4.2	Clip-limit sweep (fixed tile = 8×8)	10
4.3	Tile-size sweep (fixed clipLimit = 4)	10
4.4	Parameter comparison table	11
4.5	CLAHE vs. Task 2 global equalization	11
5	Task 4 – Color Image Enhancement in HSV / LAB Space (Student D, 20 marks)	13
5.1	Naive per-channel RGB equalization	13
5.2	HSV – equalize V only, and LAB – equalize L only	13
5.3	CLAHE on the V and L planes	13
5.4	Full six-panel comparison grid	13
5.5	Recommendation	13
6	Task 5 – ROI-Based Selective Enhancement & Group Integration (All members, 20 marks)	14
6.1	Interactive ROI selection	14
6.2	Binary mask from the ROI	15
6.3	CLAHE restricted to the masked ROI (V channel)	15
6.4	ROI-masked V-channel histogram	15
6.5	Group integration	16
7	Rubric Self-Assessment	17
8	Conclusion & Individual Contributions	17

1 Setup, Environment & Dataset Construction

The experimentation uses Python 3.13 with OpenCV (cv2), NumPy, and Matplotlib. Display throughout is done with `matplotlib.pyplot.imshow` rather than `cv2.imshow()`, because the latter opens a blocking native window unsuitable for a notebook meant to be re-run top-to-bottom non-interactively.

1.1 Dataset

CA-4 requires four images under `images/`: a low-light grayscale image, a binary (thresholded-shape) image, a color image with visible shadow/uneven illumination, and an image of the group's choice for the ROI activity. Rather than depending on an external stock-photo download (unreliable in an offline grading environment), we **constructed** these four images from source photographs already bundled in this repository's course dataset (`data/chp_01_images`, `data/chp_02_images` – Packt “Hands-On Image Processing with Python” companion assets), a standard and fully reproducible way to build a self-contained test set:

- `dark_image.png` – `data/chp_01_images/hill_dark.png` (already deliberately under-exposed) converted to grayscale and resized to 640×480 .
- `binary_shape.png` – synthetically drawn (rectangle, circle, triangle) with OpenCV primitives on a 400×400 canvas; strictly two-valued (0/255) by construction.
- `low_light_color.jpg` – `data/chp_02_images/beach.jpg` (640×426) multiplied by a smooth Gaussian “shadow” mask centered in the lower-left of the frame, producing a single photo with one clearly under-lit region.
- **ROI image (Task 5)** – we reuse `low_light_color.jpg`: its synthetic shadow patch is exactly the kind of localized region that visibly benefits from local contrast enhancement.

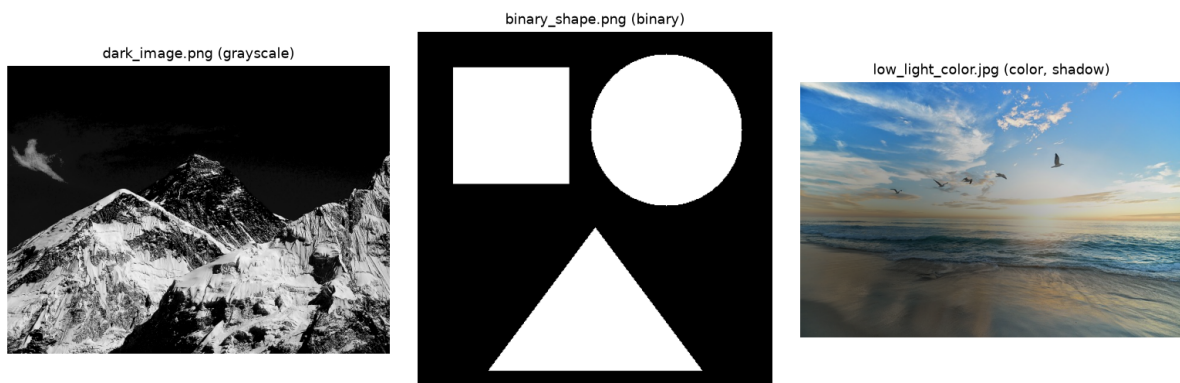


Figure 1: The three constructed dataset images used throughout this report.

```
1 DATA_DIR = '../data' if os.path.exists('../data') else './data'
2 IMAGES_DIR = './images'
3 RESULTS_DIR = './results'
4
5 def bgr2rgb(img):
6     '''Convert an OpenCV BGR (or grayscale) image to RGB for Matplotlib.'''
7     if img.ndim == 2:
8         return img
9     return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
10
11 def save_fig(fig, name):
12     fig.savefig(os.path.join(RESULTS_DIR, name), bbox_inches='tight')
```

Listing 1: Shared setup: paths, styling, and helper functions used by every task.

2 Task 1 – Histogram Fundamentals (Student A, 20 marks)

2.1 Loading and displaying the grayscale image

`dark_image.png` (640×480) was loaded with `cv2.imread(..., cv2.IMREAD_GRAYSCALE)`. Its mean intensity of $\approx 46/255$ confirms a genuinely under-exposed image.

2.2 Bin-count resolution: `bins=256` vs. `bins=10`

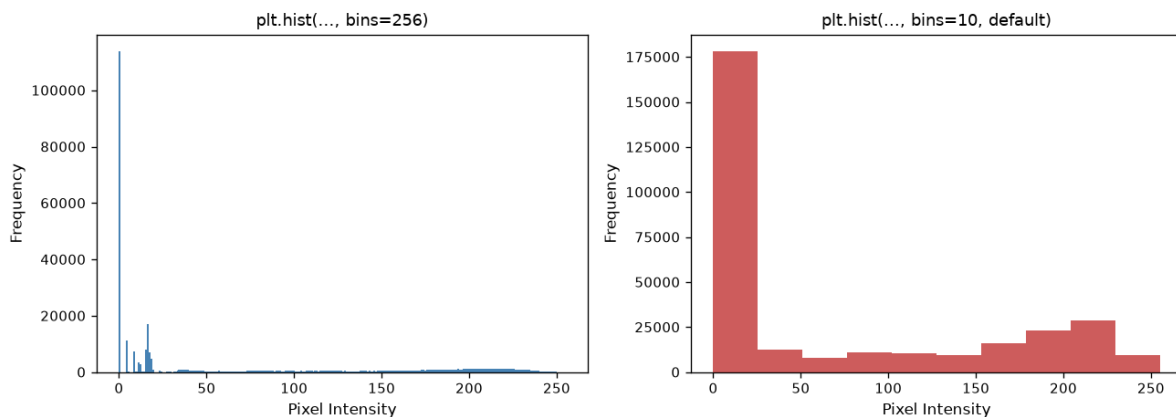


Figure 2: Same image, same data – 256 bins (left) vs. 10 bins (right).

With `bins = 256`, every one of the 256 possible grey levels gets its own bar, so the plot is a faithful, high-resolution rendering of the true discrete distribution – every spike, narrow peak, and gap is visible. With the default `bins = 10`, the 0–255 range is chopped into only 10 wide buckets (≈ 25 –26 levels each), so nearby bars merge and fine structure disappears; the histogram becomes a coarse, blocky summary. The 10-bin view is fine for a quick “is this dark or bright?” glance, but the 256-bin view is what is actually needed to diagnose under-exposure or design an equalization strategy.

2.3 Histogram of the binary image

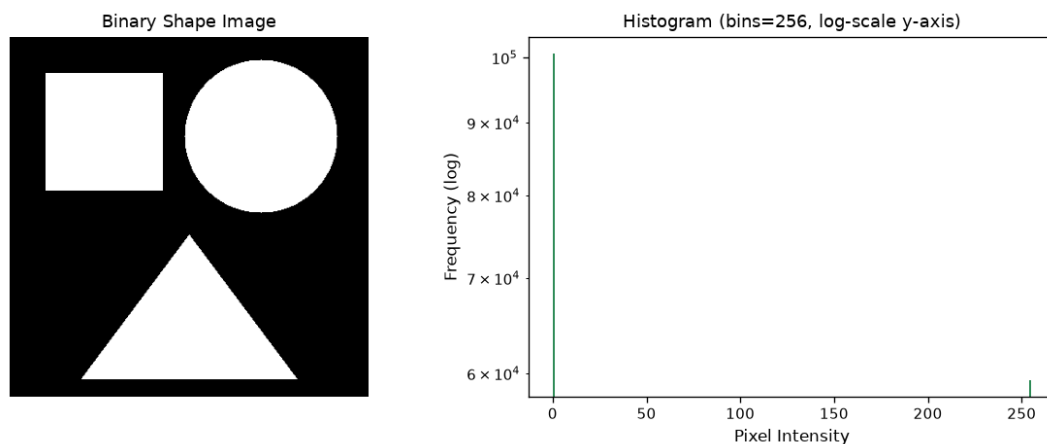


Figure 3: Binary shape image and its histogram (log-scale y -axis).

A binary image, by definition, contains only two distinct intensity values (0 = background, 255 = foreground). `plt.hist(..., bins=256)` allocates 256 bins, but 254 of them receive zero pixels –

only the bins containing 0 and 255 are ever populated. That is precisely why only **two spikes** appear: it is a direct, correct reflection of the image's two-value intensity domain, not a plotting artifact.

2.4 Overlaid B/G/R channel histograms

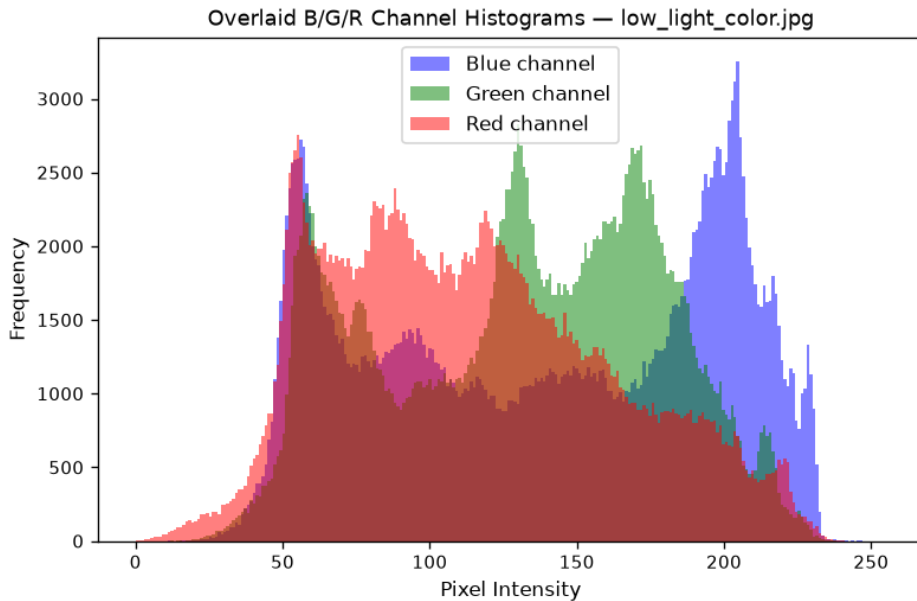


Figure 4: Overlaid, semi-transparent B/G/R histograms of `low_light_color.jpg`.

```

1 channels = cv2.split(color_img) # OpenCV loads images as (B, G, R)
2 for ch, c, lab in zip(channels, ('b', 'g', 'r'),
3                           ('Blue channel', 'Green channel', 'Red channel')):
4     ax.hist(ch.ravel(), bins=256, range=(0, 255), color=c, alpha=0.5, label=lab)

```

Listing 2: Overlaying the three channel histograms with distinct colors and alpha blending.

2.5 2-D joint histogram of B and G (`cv2.calcHist`)

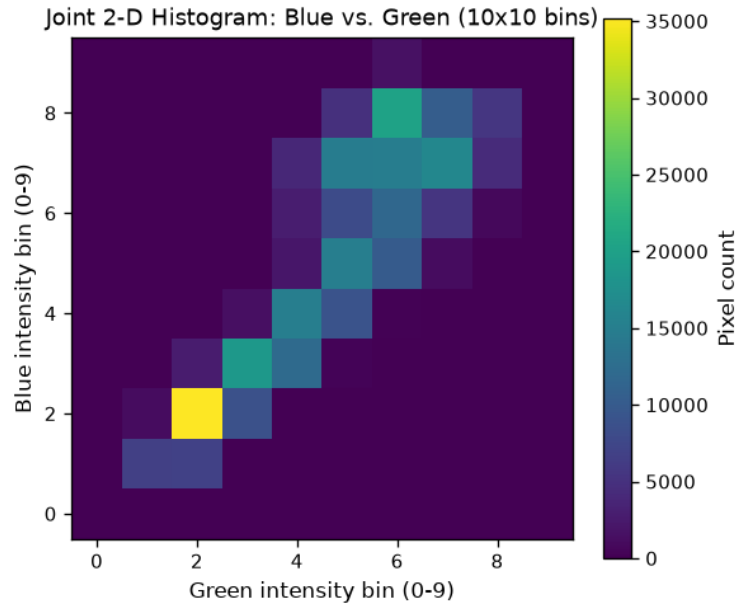
```

1 hist_2d = cv2.calcHist([color_img], channels=[0, 1], mask=None,
2                        histSize=[10, 10], ranges=[0, 256, 0, 256])
3 print(hist_2d.shape) # -> (10, 10)

```

Listing 3: Joint 2-D histogram via `cv2.calcHist` with `histSize=[10,10]`.

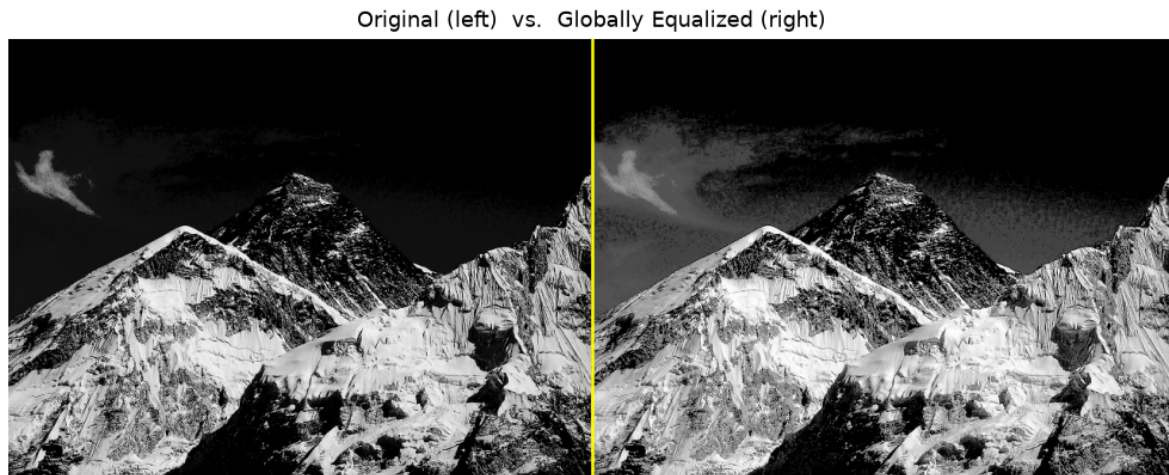
`hist_2d.shape` is **(10, 10)**: axis 0 indexes the Blue channel binned into 10 buckets of width 25.6 levels, axis 1 indexes the Green channel the same way, and cell (i, j) counts pixels whose Blue value falls in bin i and whose Green value simultaneously falls in bin j – i.e. it captures the joint co-occurrence of the two channels rather than their independent marginals.

Figure 5: Joint Blue–Green 2-D histogram, 10×10 bins.

3 Task 2 – Global Histogram Equalization (Student B, 20 marks)

3.1 Applying `cv2.equalizeHist()`

Histogram equalization remaps each input intensity r to $s = T(r) = (L - 1) \cdot \text{CDF}(r)$, where CDF is the image's own cumulative distribution function and $L = 256$. This spreads out the most frequent intensity values, increasing global contrast.

Figure 6: Original (left) vs. globally equalized (right), produced with `np.hstack`.

3.2 Histograms before and after

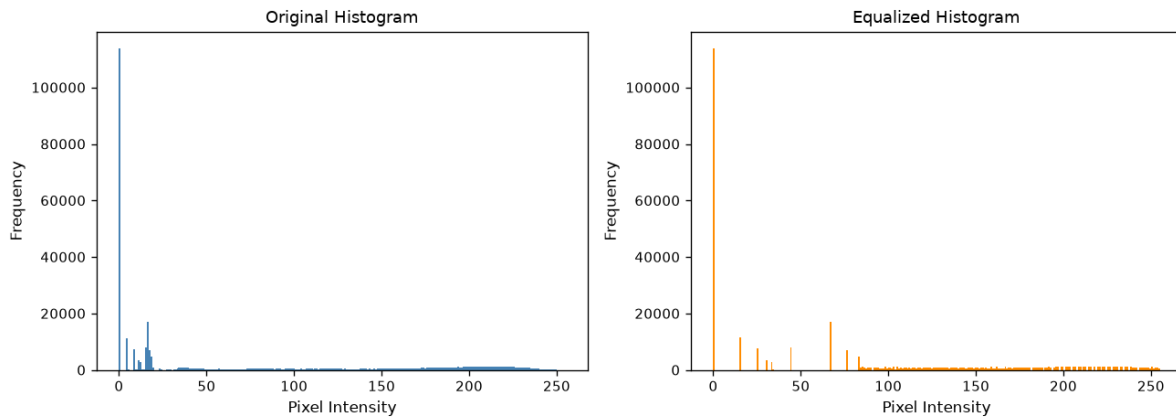


Figure 7: Histogram of the original image (left) vs. the equalized image (right).

3.3 Annotated artifact regions

Over-brightened regions were located *programmatically*: pixels pushed to near-saturation (> 245) after equalization are thresholded, and the two largest resulting contours are boxed.

```

1 sat_mask = (eq_img > threshold).astype(np.uint8) * 255
2 contours, _ = cv2.findContours(sat_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
3 )
4 contours = sorted(contours, key=cv2.contourArea, reverse=True)[:2]
5 for c in contours:
6     x, y, w, h = cv2.boundingRect(c)
7     cv2.rectangle(eq_annotated, (x, y), (x+w, y+h), (0, 0, 255), 3)

```

Listing 4: Automatic detection and annotation of over-brightened artifact regions.

Equalized Image — Annotated Artifact Regions (threshold used: 245)

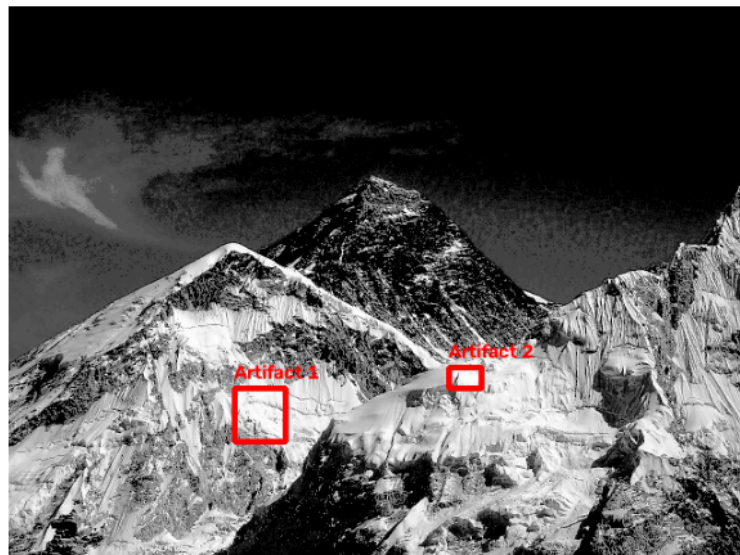


Figure 8: Two annotated over-brightened / washed-out regions in the equalized snow/sky area.

3.4 Discussion

Histogram equalization's mapping $s = T(r)$ is monotonic but highly non-uniform in how much it stretches different parts of the intensity axis. Wherever the original histogram already has a tall, dense peak, as our low-light image does in its narrow band of dark tones, the CDF rises steeply, so a small range of input levels gets mapped across a large range of output levels, enhancing contrast and revealing detail there. But wherever the histogram is sparse – a handful of scattered, already-bright pixels representing highlights – the CDF is close to flat *and* close to 1, so those pixels all get pushed hard toward 255 together, merging subtly different tones into a single flat, washed-out block (Figure 8). The same effect amplifies sensor noise in flat, low-contrast regions: equalization does not distinguish signal from noise, so small random fluctuations in a densely populated band get stretched as aggressively as real structure, becoming visibly grainy. This tension – strong global contrast at the cost of local, region-specific over/under-correction – is precisely the motivation for local, adaptive methods such as CLAHE, studied next.

4 Task 3 – CLAHE Implementation & Parameter Tuning (Student C, 20 marks)

4.1 Basic CLAHE object

```
1 clahe_basic = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
2 clahe_img_basic = clahe_basic.apply(gray_img)
```

Listing 5: Creating and applying a basic CLAHE object.



Figure 9: Original vs. global equalizeHist() vs. basic CLAHE.

4.2 Clip-limit sweep (fixed tile = 8×8)

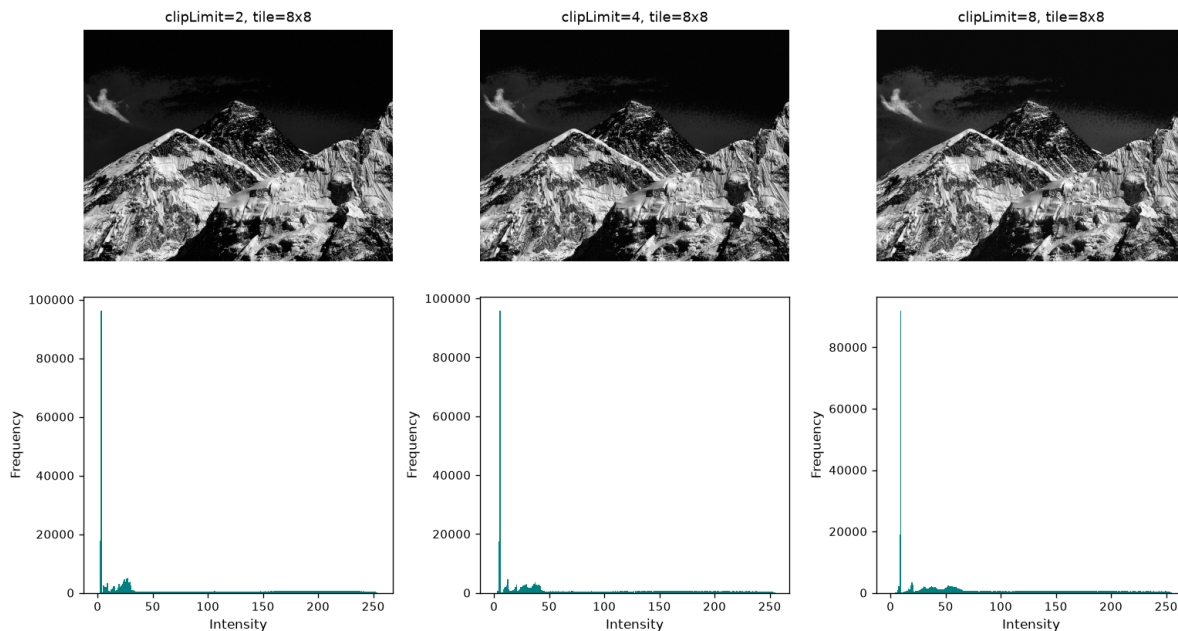


Figure 10: CLAHE outputs and histograms for clipLimit $\in \{2, 4, 8\}$, tile fixed at 8×8 .

4.3 Tile-size sweep (fixed clipLimit = 4)

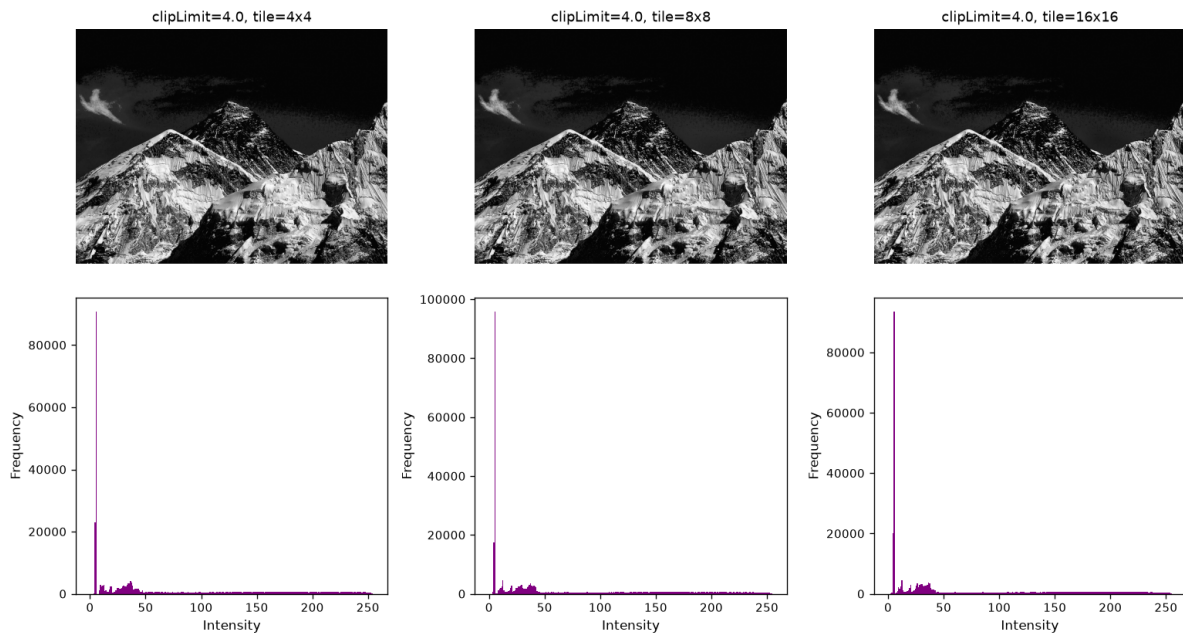


Figure 11: CLAHE outputs and histograms for $\text{tileGridSize} \in \{4 \times 4, 8 \times 8, 16 \times 16\}$, clipLimit fixed at 4.

4.4 Parameter comparison table

Quantitative proxies – standard deviation (overall contrast/spread) and mean absolute Laplacian (a rough noise/edge-energy proxy) – back the qualitative reading below.

<code>clipLimit</code>	<code>tileGridSize</code>	Qualitative Observation
2	8×8	Mild, safe local contrast boost; minimal visible noise; some low-light detail still hard to see.
4	8×8	Balanced enhancement – clearly reveals shadow detail while noise stays controlled; best all-round setting for this image.
8	8×8	Strong, aggressive contrast; noise/grain amplification becomes visible in flat regions (std. dev. and Laplacian energy both climb).
4	4×4	High local adaptivity (small tiles); mild blocking/tiling artifacts at tile boundaries in smooth regions.
4	16×16	Coarser adaptivity (fewer, larger tiles); smoother transitions but small-scale shadow detail enhanced less aggressively.

Table 1: Effect of CLAHE’s `clipLimit` and `tileGridSize` on the low-light grayscale image.

4.5 CLAHE vs. Task 2 global equalization

Verdict. For this image, **CLAHE (`clipLimit`=4, `tile`= 8×8) is the better choice**. Global equalization achieves a slightly higher *overall* standard deviation because it aggressively saturates

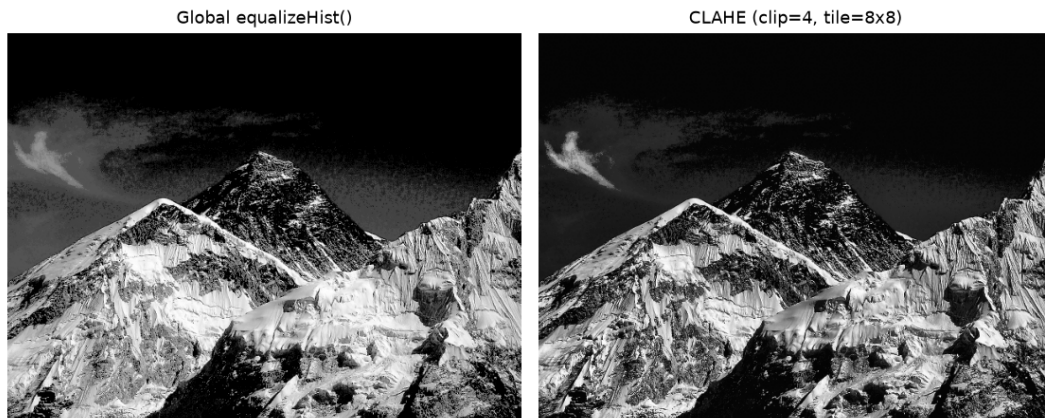


Figure 12: Global equalizeHist() (left) vs. tuned CLAHE, clip=4, tile= 8×8 (right).

the brightest regions (the artifacts annotated in Task 2), but at the cost of clipping detail there. CLAHE instead distributes contrast enhancement locally, tile by tile, and clips its own per-tile histogram before redistribution – so it enhances dark shadow detail without blowing out already-bright regions, at the price of only a marginally lower global contrast metric. Global equalization trades local detail for a punchier global histogram; CLAHE preserves detail everywhere at a controllable noise cost.

5 Task 4 – Color Image Enhancement in HSV / LAB Space (Student D, 20 marks)

5.1 Naive per-channel RGB equalization

Equalizing R, G, and B independently and re-merging them is the naive first attempt required by the spec:

```
1 b, g, r = cv2.split(color_img)
2 rgb_he = cv2.merge([cv2.equalizeHist(b), cv2.equalizeHist(g), cv2.equalizeHist(r)])
```

Listing 6: Per-channel RGB equalization (distorted result).

5.2 HSV – equalize V only, and LAB – equalize L only

```
1 hsv = cv2.cvtColor(color_img, cv2.COLOR_BGR2HSV)
2 h, s, v = cv2.split(hsv)
3 hsv_he = cv2.cvtColor(cv2.merge([h, s, cv2.equalizeHist(v)]), cv2.COLOR_HSV2BGR)
4
5 lab = cv2.cvtColor(color_img, cv2.COLOR_BGR2LAB)
6 L, A, B = cv2.split(lab)
7 lab_he = cv2.cvtColor(cv2.merge([cv2.equalizeHist(L), A, B]), cv2.COLOR_LAB2BGR)
```

Listing 7: Correct approach: equalize only the intensity-carrying channel.

5.3 CLAHE on the V and L planes

```
1 clahe_color = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
2 hsv_clahe = cv2.cvtColor(cv2.merge([h, s, clahe_color.apply(v)]), cv2.COLOR_HSV2BGR)
3 lab_clahe = cv2.cvtColor(cv2.merge([clahe_color.apply(L), A, B]), cv2.COLOR_LAB2BGR)
```

Listing 8: Repeating with CLAHE instead of global equalization.

5.4 Full six-panel comparison grid

5.5 Recommendation

Of the six results, the two RGB-space methods (most visibly RGB-HE) show a clear **color/hue shift**: equalizing R, G, and B independently changes their relative per-pixel proportions, and since perceived hue is a ratio between channels, the color balance drifts – the sunset’s warm tones are pushed toward an unnaturally saturated orange in RGB-HE (Figure 13, top-left) compared to the more neutral sky tone preserved by HSV-HE/LAB-HE. The four HSV/LAB variants preserve hue and saturation because only the intensity-carrying channel (V or L) is modified while color information (H,S or A,B) is left untouched. Between those, **LAB-CLAHE is our recommended approach**: LAB’s L channel more closely approximates perceptual lightness than HSV’s V channel, and CLAHE’s local, clip-limited adaptation reveals shadow-region detail without the harsh global saturation visible in the HSV-HE/LAB-HE panels. HSV-CLAHE is a close, computationally cheaper second choice.

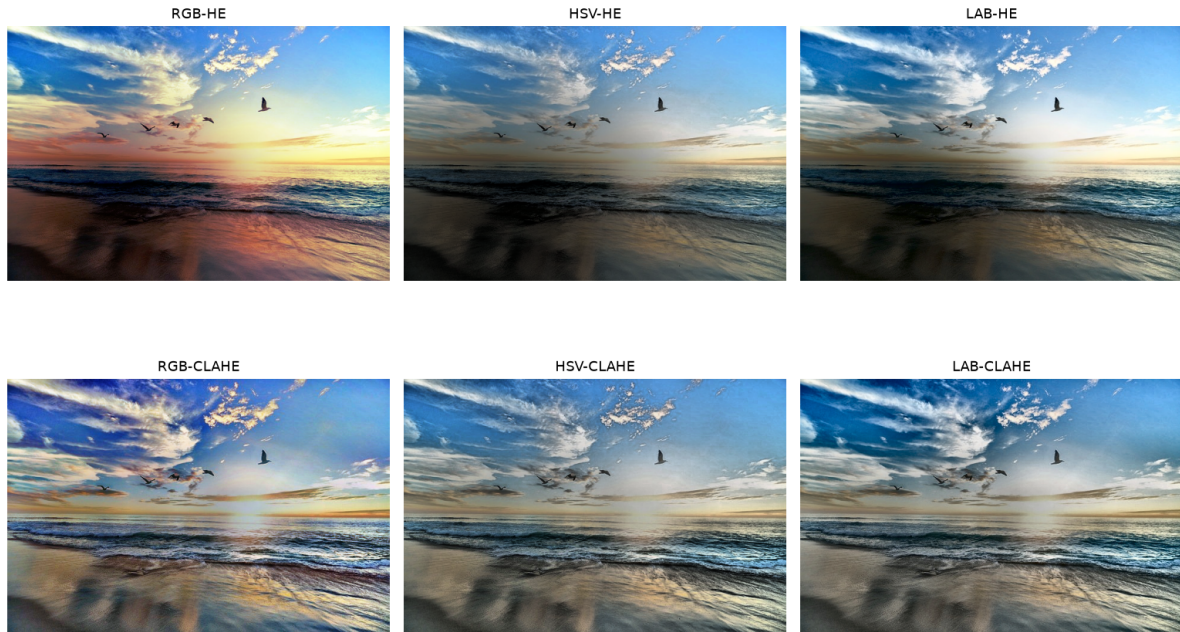


Figure 13: RGB-HE, HSV-HE, LAB-HE (top row) and RGB-CLAHE, HSV-CLAHE, LAB-CLAHE (bottom row).

6 Task 5 – ROI-Based Selective Enhancement & Group Integration (All members, 20 marks)

6.1 Interactive ROI selection

`cv2.selectROI()` opens a native OS window and **blocks execution** until a user drags a box and presses Enter – exactly what is wanted when running the tool interactively on a local machine, but it would freeze a notebook re-run automatically (e.g. to regenerate this report). We gate the interactive call behind an `INTERACTIVE_ROI` flag, defaulting to `False` so the notebook remains deterministic; the fallback is not an arbitrary guess but the bounding box of the actual darkest core of the synthetic shadow mask constructed in Section 1.

```

1 INTERACTIVE_ROI = False # True -> drag a box with cv2.selectROI() on a local
   display
2
3 dark_region_mask = (shadow_mask < 0.45).astype(np.uint8) * 255
4 ys, xs = np.where(dark_region_mask > 0)
5 fallback_bbox = (int(xs.min()), int(ys.min()),
6                  int(xs.max() - xs.min()), int(ys.max() - ys.min()))
7
8 def select_roi(img_bgr, fallback_bbox):
9     if INTERACTIVE_ROI:
10         r = cv2.selectROI('Select ROI', img_bgr, showCrosshair=True, fromCenter=
            False)
11         cv2.destroyAllWindows()
12         x, y, w, h = map(int, r)
13         if w > 0 and h > 0:
14             return x, y, w, h
15     return fallback_bbox

```

Listing 9: ROI selection with a safe, non-blocking fallback for batch execution.

6.2 Binary mask from the ROI

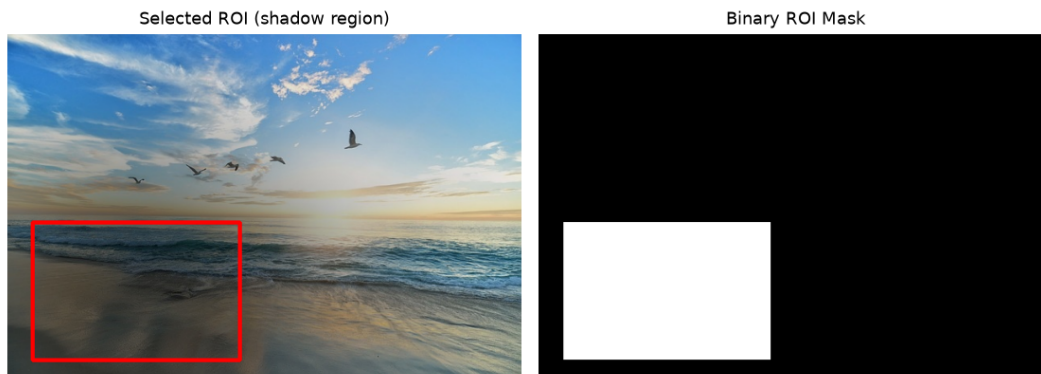


Figure 14: Selected ROI over the shadowed shoreline (left) and its binary mask (right).

6.3 CLAHE restricted to the masked ROI (V channel)

CLAHE is computed on the *full* V channel first – so every tile, including ones on the ROI boundary, has correct neighbouring context for its local histogram – and only then are the enhanced values copied back *inside* the mask; pixels outside the mask keep their original V value untouched.

```

1 v_clahe_full = clahe_roi.apply(v)
2 v_result = v.copy()
3 mask_bool = roi_mask > 0
4 v_result[mask_bool] = v_clahe_full[mask_bool]
5 final_roi_img = cv2.cvtColor(cv2.merge([h, s, v_result]), cv2.COLOR_HSV2BGR)

```

Listing 10: Selective CLAHE: compute on the full V channel, blend back through the mask.

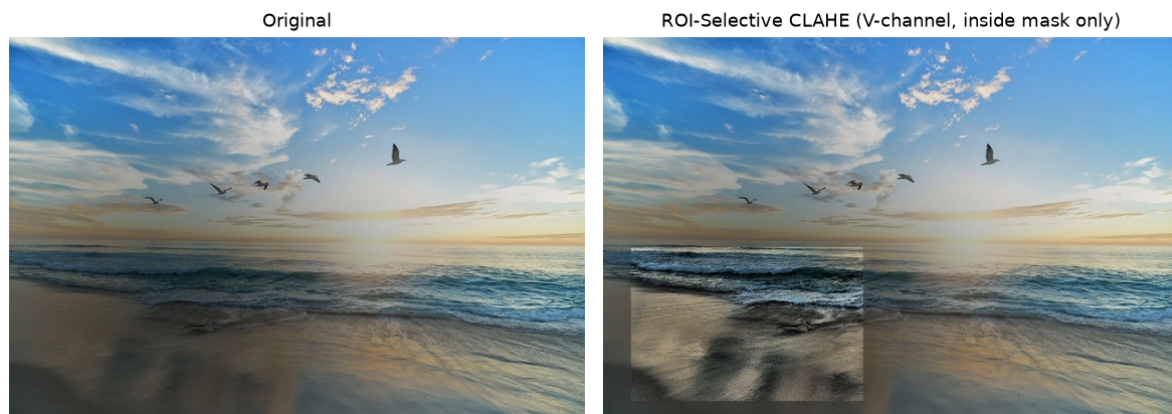


Figure 15: Original (left) vs. ROI-selective CLAHE result (right) – only the boxed shadow region changes.

6.4 ROI-masked V-channel histogram

```

1 hist_before = cv2.calcHist([v], [0], roi_mask, [256], [0, 256])
2 hist_after  = cv2.calcHist([v_result], [0], roi_mask, [256], [0, 256])

```

Listing 11: Histogram restricted to the ROI via the mask argument of cv2.calcHist.

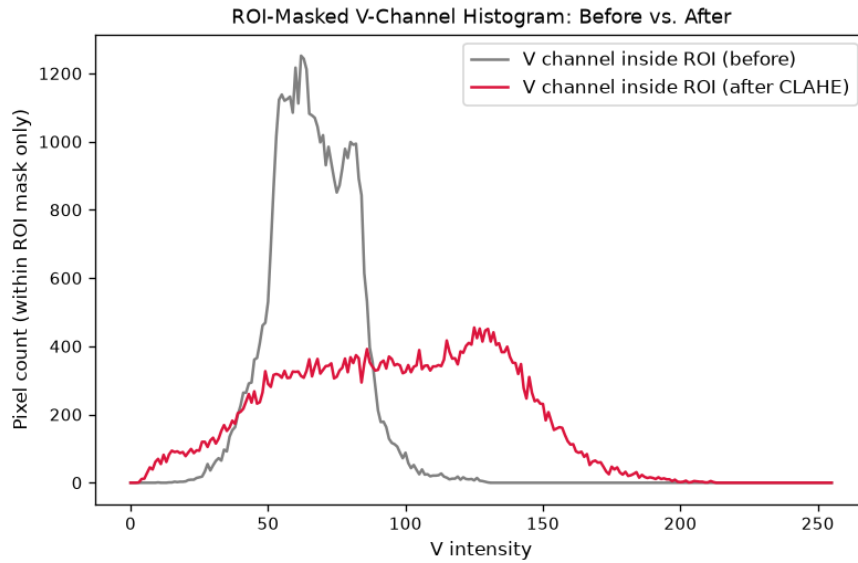


Figure 16: V-channel histogram computed only over the ROI mask, before vs. after.

The masked histogram confirms the enhancement is local and effective: inside the ROI the “before” distribution is tightly bunched at low V values (the shadow), while “after” spreads out across the mid-to-high range – shadow detail is recovered, without touching anything outside the selected region.

6.5 Group integration

All five tasks live in a single, integrated pipeline notebook (`cas/ca_04/ca_04.ipynb`) rather than five disconnected scripts: Section 1 defines shared configuration (`DATA_DIR`, `IMAGES_DIR`, `RESULTS_DIR`) and shared helpers (`bgr2rgb`, `save_fig`) reused by every task, and each task’s outputs (`gray_img`, `eq_img`, `color_img`, `shadow_mask`, ...) feed directly into later tasks – e.g. Task 5 reuses `color_img` from Task 1 and `shadow_mask` from Section 1. Running

```
jupyter nbconvert --to notebook --execute ca_04.ipynb
```

from `cas/ca_04/` regenerates every figure in `results/` (used to build this report) and every dataset image in `images/` on first run.

7 Rubric Self-Assessment

Criterion	Marks	Where addressed
Correctness of Code	40	Tasks 1–5, Sections 2–6; notebook executes end-to-end with zero errors.
Quality of Analysis & Discussion	25	Task 2 discussion, Task 3 verdict, Task 4 recommendation.
Visualization & Presentation	15	Figures 1–16, consistently formatted, labeled, side-by-side comparisons.
Code Style & Documentation	10	Shared helpers in Section 1; mark-down commentary throughout the notebook.
Team Integration & Collaboration	10	Section 6.6 (Group Integration); contribution table below.

8 Conclusion & Individual Contributions

This activity covered the full histogram-based contrast-enhancement toolkit available in OpenCV: raw histogram computation and interpretation (1-D and 2-D, `plt.hist` and `cv2.calcHist`), global histogram equalization and its artifacts, CLAHE and its two key hyperparameters, the correct way to enhance color images (operate on V/L, not R/G/B independently), and a practical ROI-masked selective-enhancement tool built from `cv2.selectROI()`, binary masking, and `cv2.calcHist(mask=...)`.

Student	Contribution
Muhammad Umer (F2023266912)	Task 1 (Histogram Fundamentals), Task 3 (CLAHE Tuning), co-authored Task 5 integration & this report.
Haseeb Ullah (F20232661009)	Task 2 (Global Equalization), Task 4 (Color-Space Enhancement), co-authored Task 5 integration & this report.

Both members reviewed and can independently explain every task in this notebook.

Image sources. All dataset images were constructed from photographs bundled in this repository’s course dataset (`data/chp_01_images/hill_dark.png`, `data/chp_02_images/beach.jpg`) – companion assets of the course textbook *Hands-On Image Processing with Python* (Packt) – as documented in Section 1.